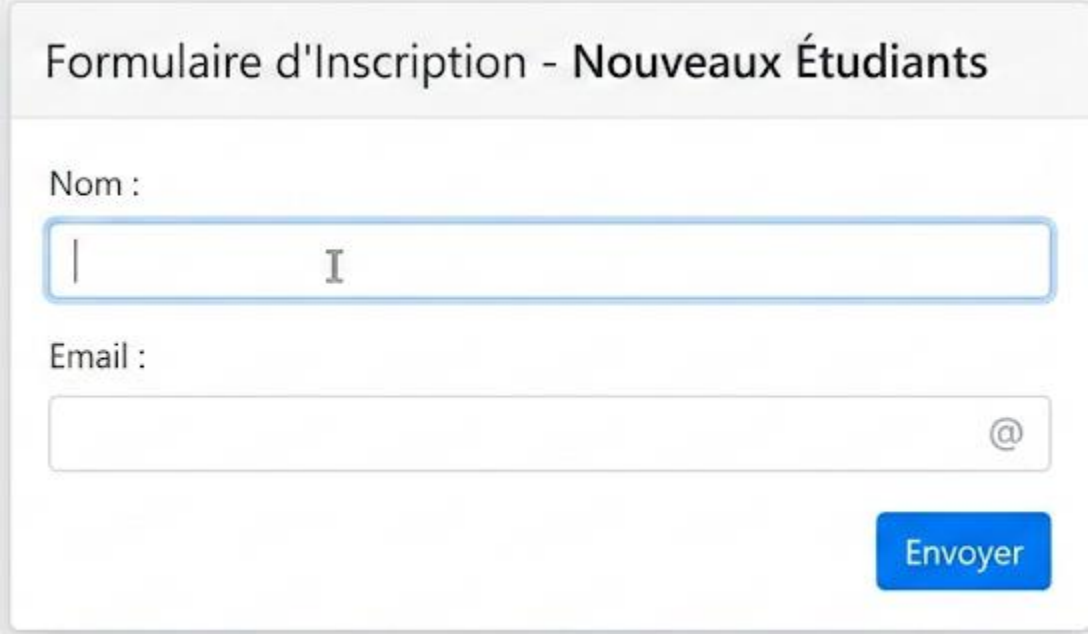
# Partie 5 : Formulaires & Validation

## Formulaire HTML

*Un formulaire HTML permet d'envoyer des données vers le serveur.*

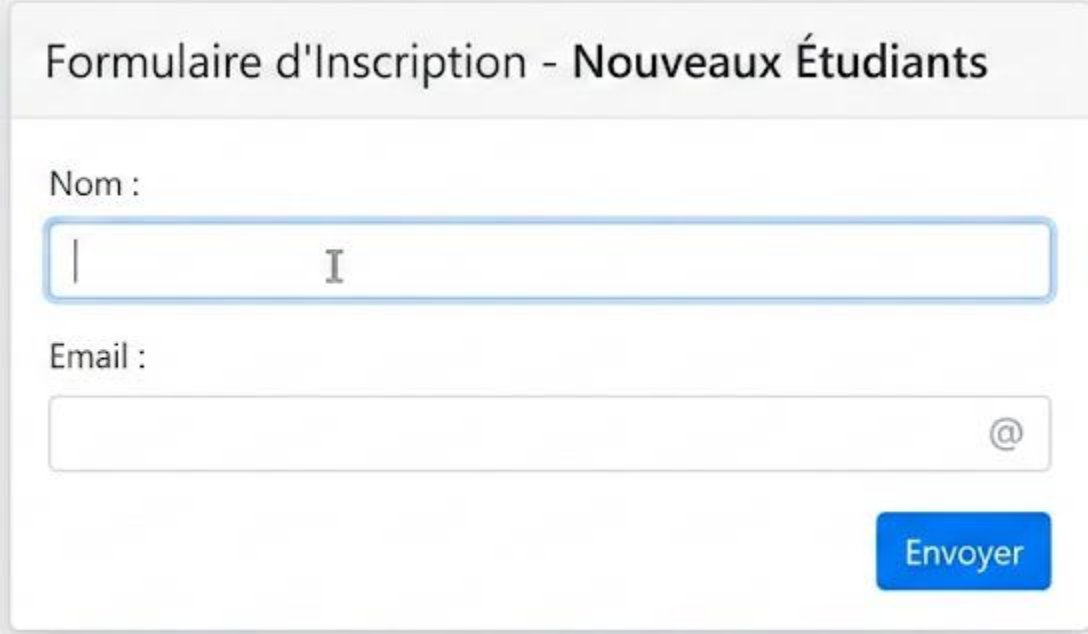
```html
<form action="/etudiants" method="POST">
    <label>Nom :</label>
    <input type="text" name="nom">

    <label>Email :</label>
    <input type="email" name="email">

    <button type="submit">Envoyer</button>
</form>
```



# 5. Formulaires & Validation

## Formulaire HTML avec Blade

```
@extends('layouts.app')

@section('content')

<h2>Ajouter un étudiant</h2>

<form action="{{ route('students.store') }}" method="POST">
    @csrf

    <div>
        <label>Nom :</label>
        <input type="text" name="name" value="{{ old('name') }}">
    </div>

    <div>
        <label>Email :</label>
        <input type="email" name="email" value="{{ old('email') }}">
    </div>

    <div>
        <label>Age :</label>
        <input type="number" name="age" value="{{ old('age') }}">
    </div>

    <button type="submit">Enregistrer</button>
</form>

@endsection
```



# 5. Formulaires & Validation

**Request Laravel**

***Lorsqu'un formulaire est soumis, Laravel récupère les données via l'objet Request***

```php
class UserController extends Controller
{
    public function liste_users(Request $rq){
        dd($rq);
    }
}
```

```
Illuminate\Http\Request {#52 ▼ // app\Http\Controllers\UserController.php:10
  +attributes: Symfony\Component\HttpFoundation\ParameterBag {#57 ▶}
  +request: Symfony\Component\HttpFoundation\InputBag {#53 ▶}
  +query: Symfony\Component\HttpFoundation\InputBag {#60 ▶}
  +server: Symfony\Component\HttpFoundation\ServerBag {#55 ▶}
  +files: Symfony\Component\HttpFoundation\FileBag {#59 ▶}
  +cookies: Symfony\Component\HttpFoundation\InputBag {#58 ▶}
  +headers: Symfony\Component\HttpFoundation\HeaderBag {#54 ▶}
  #content: null
  #languages: null
  #charsets: null
  #encodings: null
  #acceptableContentTypes: null
  #pathInfo: "/users"
  #requestUri: "/users"
  #baseUrl: ""
  #basePath: null
  #method: "GET"
  #format: null
  #session: Illuminate\Session\SymfonySessionDecorator {#256 ▶}
  #locale: null
  #defaultLocale: "en"
  -preferredFormat: null
  -isHostValid: true
  -isForwardedValid: true
  -isSafeContentPreferred: ? bool
  -trustedValuesCache: []
  -isIisRewrite: false
  #json: null
  #convertedFiles: null
  #userResolver: Closure($guard = null) {#214 ▶}
  #routeResolver: Closure() {#226 ▶}
  #cachedAcceptHeader: null
  basePath: ""
  format: "html"
}
```

## Méthode utiles

| Méthode | Description |
|---|---|
| $request->all() | Toutes les données |
| $request->input('name') | Récupérer un champ |
| $request->has('name') | Vérifier l'existence |
| $request->only([...]) | Champs spécifiques |
| $request->except([...]) | Exclure des champs |

# Validation des données

***Laravel permet de valider facilement les données.***

```php
public function store(Request $request)
{
    $validator = Validator::make($request->all(),[
        'name' => 'required|min:3|max:100',
        'email' => 'required|email',
    ],[
        'name.required' => 'Le nom est obligatoire.',
        'name.min'      => 'Le nom doit contenir au moins 3 caractères.',
        'name.max'      => 'Le nom ne peut pas dépasser 100 caractères.',
        'email.required'=> 'L\'email est obligatoire.',
        'email.email'   => 'Veuillez entrer un email valide.',
    ]);
    if($validator->fails()) {
        $errors = $validator->errors();
        return redirect()->back()
                        ->withErrors($errors)
                        ->withInput();
    }
    return redirect()->back()->with('success', 'Étudiant ajouté avec succès');
}
```

```php
use Illuminate\Support\Facades\Validator;
```

https://laravel.com/docs/12.x/validation

*Cette section du Blade permet d'afficher tous les messages d'erreurs de validation afin que l'utilisateur puisse voir quelles informations doivent être corrigées.*

```
@if(session('success'))
    <div style="color: green;">
        {{session('success')}}
    </div>
@endif

@if ($errors->any())
    <div>
        <ul>
            @foreach ($errors->all() as $error)
                <li style="color: red">{{$error}}</li>
            @endforeach
        </ul>
    </div>
@endif
```

# 4. Views avec Blade

## Mini TP : Interface dynamique avec Blade

### Objectif :

- Créer un formulaire avec Blade
- Valider les données avec Validator
- Afficher les erreurs de validation
- Afficher un message de succès

### Etapes :

On veut créer une *page d'inscription d'étudiant* avec : Nom, Email et Âge
Avec validation sécurisée et affichage des erreurs.

1. Créer le Contrôleur avec validation
2. Créer le Layout
3. Créer la vue formulaire et afficher les messages de succès et d'erreur.
4. Créer les Routes